

Shuttle Bot: The Future of Autonomous Sports Robotics	
Introduction	
In this assignment, your team will design and develop an autonomous robot capable of detecting, collecting, and placing shuttlecocks at a predefined location. The robot will utilize machine learning (ML) algorithms for shuttlecock detection and control techniques for maneuvering, collection, and precise placement. This project will combine your knowledge of robotics, control systems, and machine learning to build a practical solution for automating a common task in sports environments.	
Objective:	
The objective of this assignment is to challenge your team to design, construct, and program an autonomous robot that can: • Detect shuttlecocks on the ground using a machine learning-based vision system. • Navigate autonomously within a specified area, avoiding obstacles. • Pick up the shuttlecock using a mechanical arm or collection mechanism. • Transport the shuttlecock to a designated collection area and place it accurately. • Perform all tasks efficiently while maintaining stability and precision.	
FAST-NU Spring 2026	
Subject: ML for Robotics (CS-4090) Instructor: Dr. Maryam Iqbal Marks (20) Assignment# 01 (CLO-1) Class: BCS 8 A Date: 16th February 2026 Due Date: 22nd February 2026	
Automate, Navigate, Dominate! Instructions & Deliverables	
1. Form a team of 3-4 students. 2. Select the components required for the shuttle bot. 3. Conduct a financial analysis and present an estimate of the total cost of the shuttle-bot. 4. Prepare a report on the design of the shuttle bot. The report should include the following sections: • Introduction • Materials and Methods • Circuit Diagram • Financial Analysis	
Grading Rubrics The assignment will be graded based on the following criteria: 1. Design and Innovation (Marks=5) • Creativity in the design of the robot's structure and collection mechanism. • Use of cutting-edge ML and control techniques for shuttlecock detection and navigation. 2. Component Specifications (Marks=5) • List of components required for optimal performance. • Specifications of all components 3. Financial Analysis: (Marks=5) • A detailed financial breakdown of the project, including the cost of materials, components, and any additional expenses incurred during the design and construction process. 4. Teamwork (Marks=5) • Effective collaboration within the team. • Leadership and communication skills of the captain.	
You can check out this video that demonstrates an autonomous robot picking up shuttlecocks in action. It shows how the robot detects, collects, and transports shuttlecocks autonomously perfect for inspiration and understanding the mechanisms involved in your own project. Here's the link to the video: Autonomous Robot Picking Shuttlecocks	
this is my project actually i have done this report but now the teacher say due to lack of components we can't do the hardware now we have to move towards purely ML and now the project includes the simulations advanced simulations of the robot detecting shuttle and pick it up and ML code for detection of shuttle so instead of using the hardware we use simulations so when the robot saw the shuttle it detects by using simulated sensors and cameras etc modules and pick it up and collect it in the basket at the back of the robot by using its arms now step by step guide me from where we start . make sure only one step at a time after done that step then tell mw the other step . just for now we talk about the dataset should we use supervised learning or unsupervised first of just let me know this	

Thought for a couple of seconds >

For **your Shuttle Bot ML simulation**, start with **supervised learning**.

Unsupervised learning is **not the right first choice** here. Your robot needs to know:

“Is there a shuttlecock in this camera frame?”

“Where exactly is it?”

“How close is it?”

“Should the arm move to pick it?”

For this, the model must be trained on images where the shuttlecock is already labelled. That means **supervised object detection**.

Step 1: Dataset Decision

You need a dataset of shuttlecock images with labels.

Best approach:

Use Supervised Object Detection

Use models like:

YOLOv8 / YOLOv11 / SSD / Faster R-CNN

For your project, I would strongly recommend:

YOLOv8 or YOLOv11

Because it is easier, fast, accurate, and works well for robotics simulation.

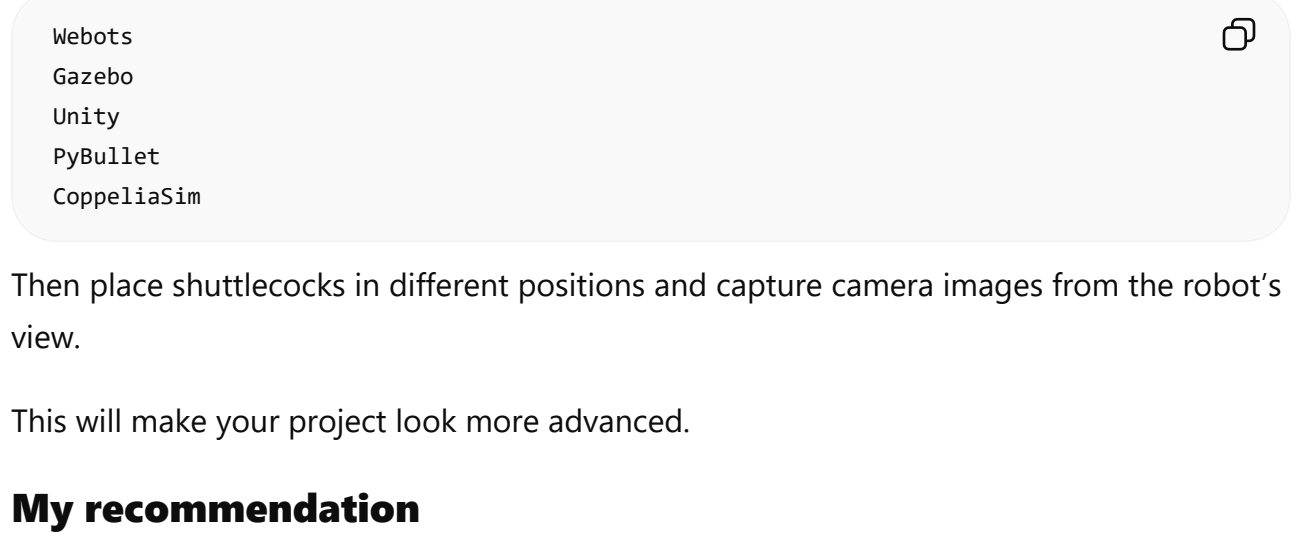
What type of dataset do you need?

You need images like:

1. Shuttlecock on badminton court
2. Shuttlecock on floor
3. Shuttlecock from different angles
4. Shuttlecock far and near
5. Shuttlecock partially hidden
6. Multiple shuttlecocks in one image
7. Different lighting conditions
8. Robot camera view/simulation camera view

Each image should have a **bounding box** around the shuttlecock.

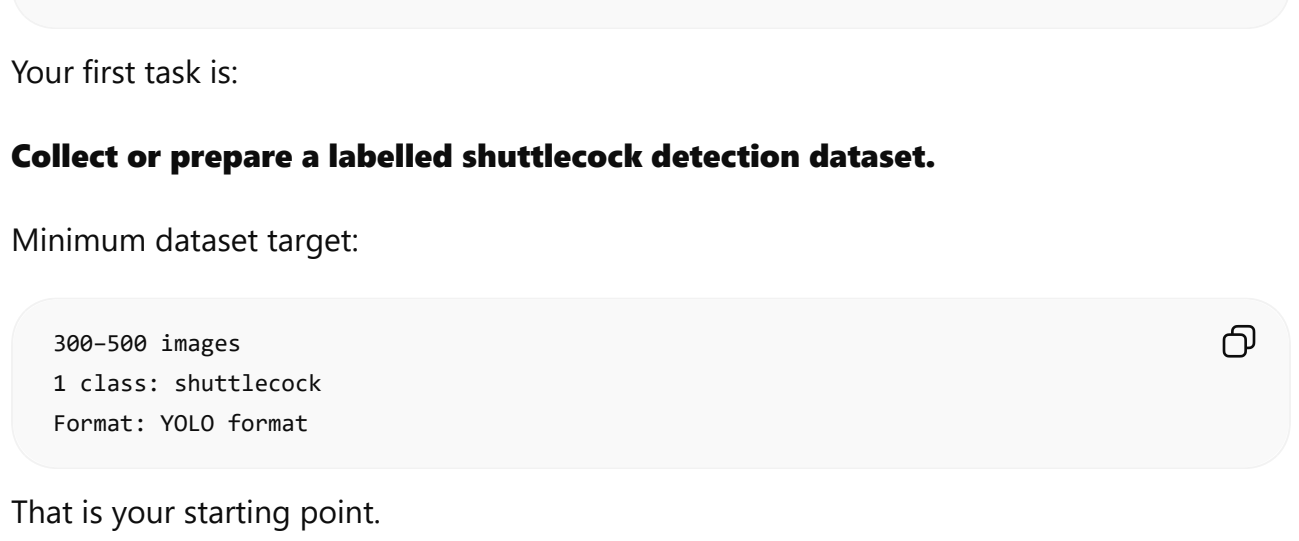
Example:



Why not unsupervised learning?

Unsupervised learning finds patterns without labels. But your robot does not need "pattern grouping." It needs **clear detection**.

Using unsupervised learning here would be a weak choice because it will not directly tell the robot:



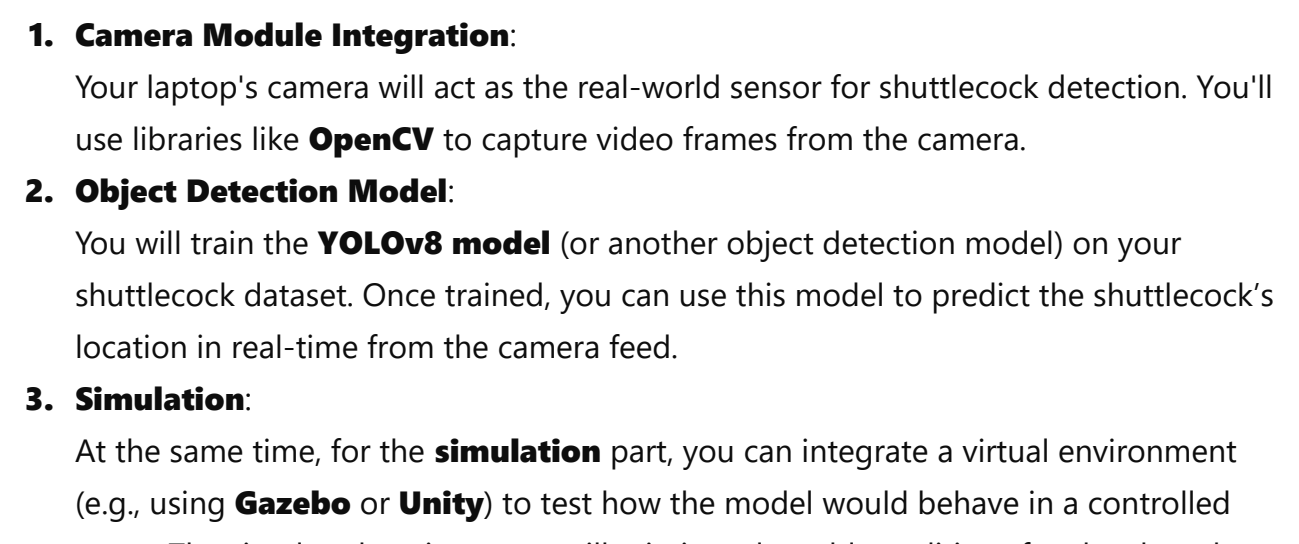
So don't overcomplicate it. Use **supervised learning**.

Dataset Options

You have 3 practical options:

Option 1: Use an existing shuttlecock dataset

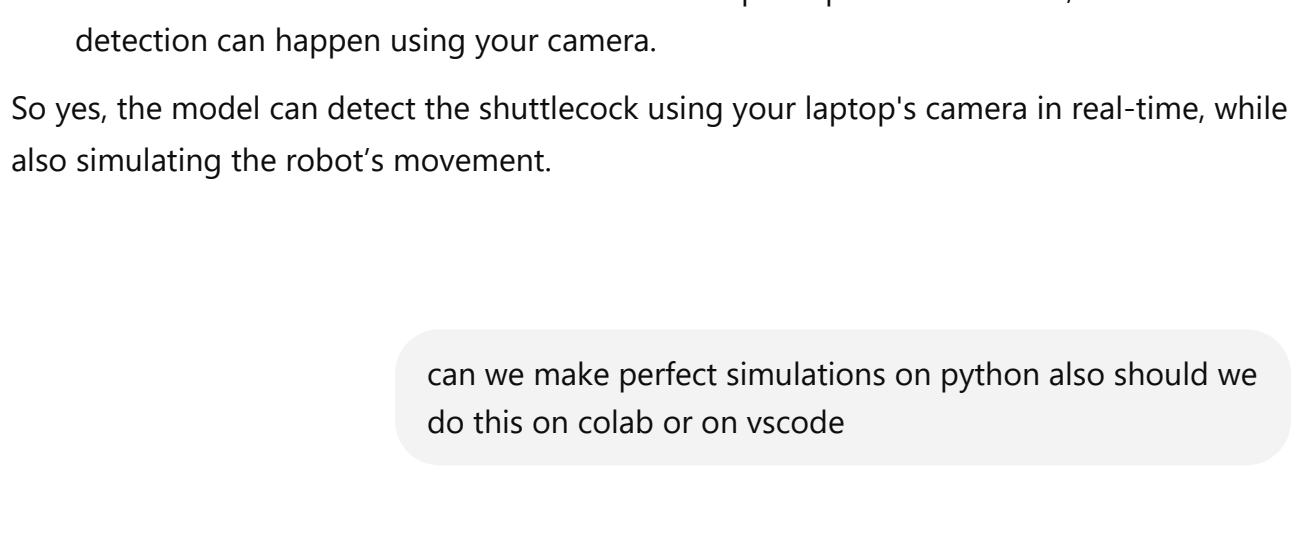
Search on:



This is the fastest option.

Option 2: Create your own dataset

Take 200-500 images of shuttlecocks from different angles and label them manually using:

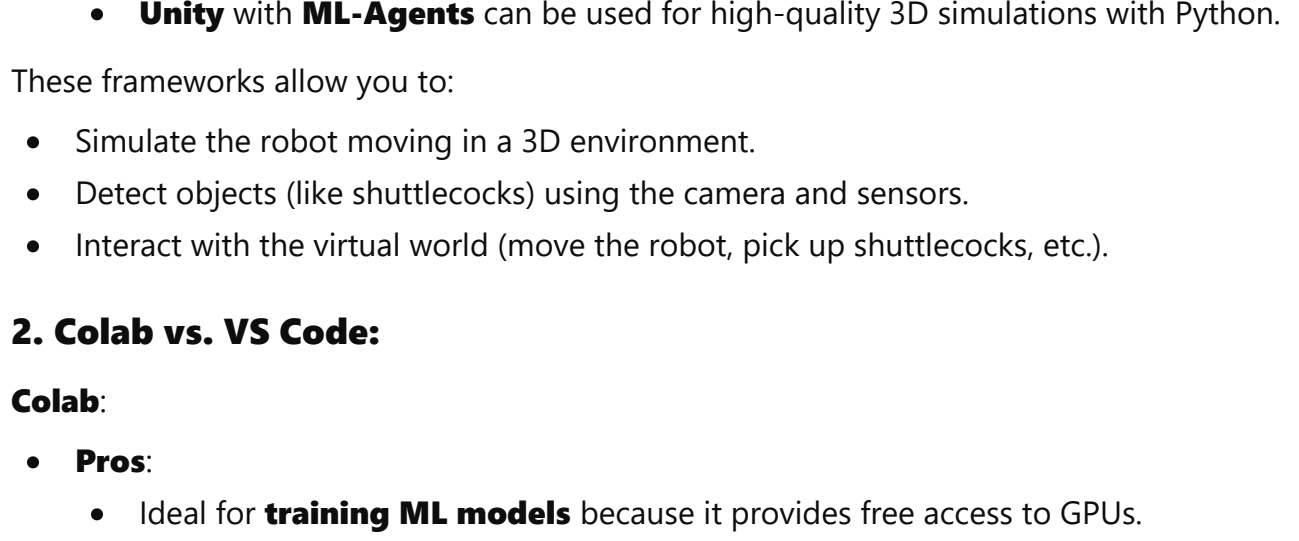


This gives better control.

Option 3: Generate simulation dataset

Since your project is now simulation-based, this is a strong idea.

You can create a virtual badminton court in:

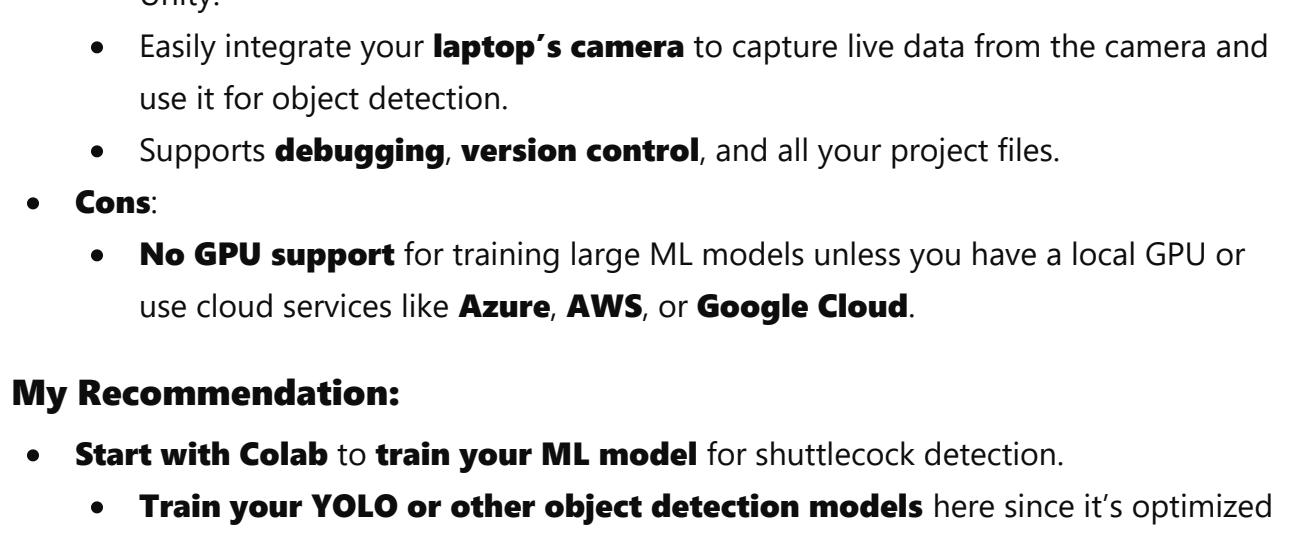


Then place shuttlecocks in different positions and capture camera images from the robot's view.

This will make your project look more advanced.

My recommendation

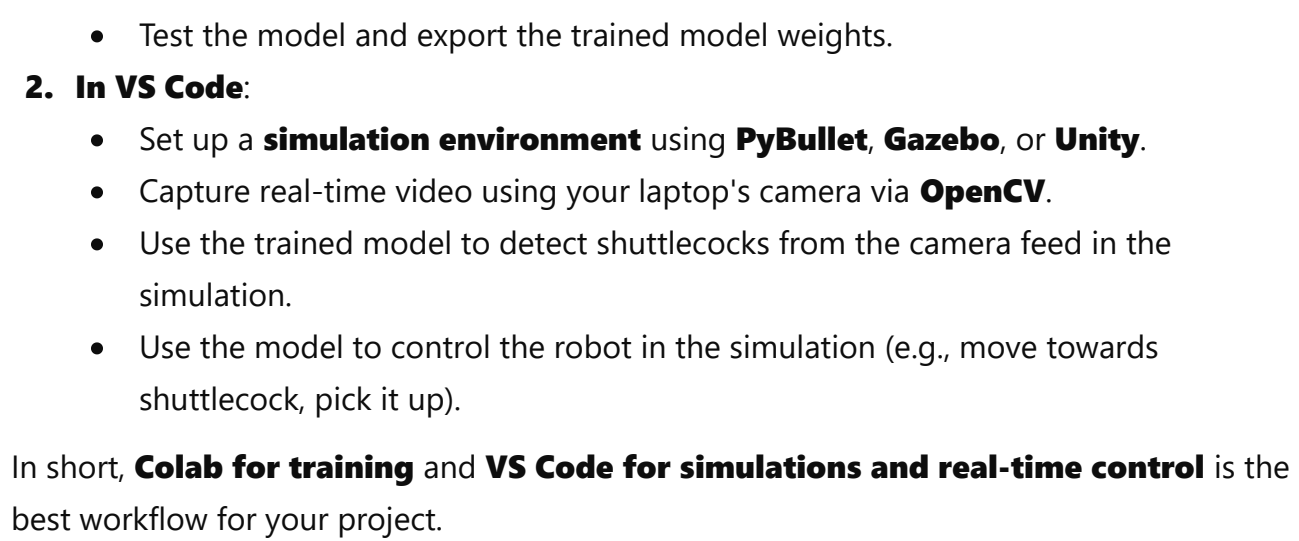
Use a **hybrid dataset**:



This is the best choice because your model will learn both real-world appearance and simulated environment appearance.

Final decision for Step 1

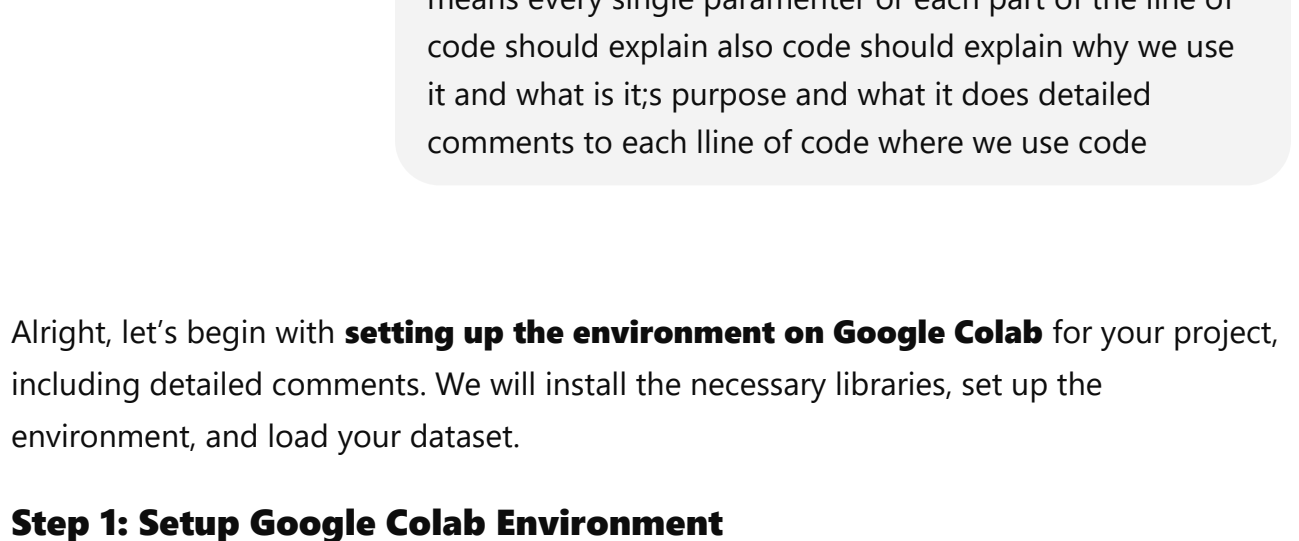
Use:



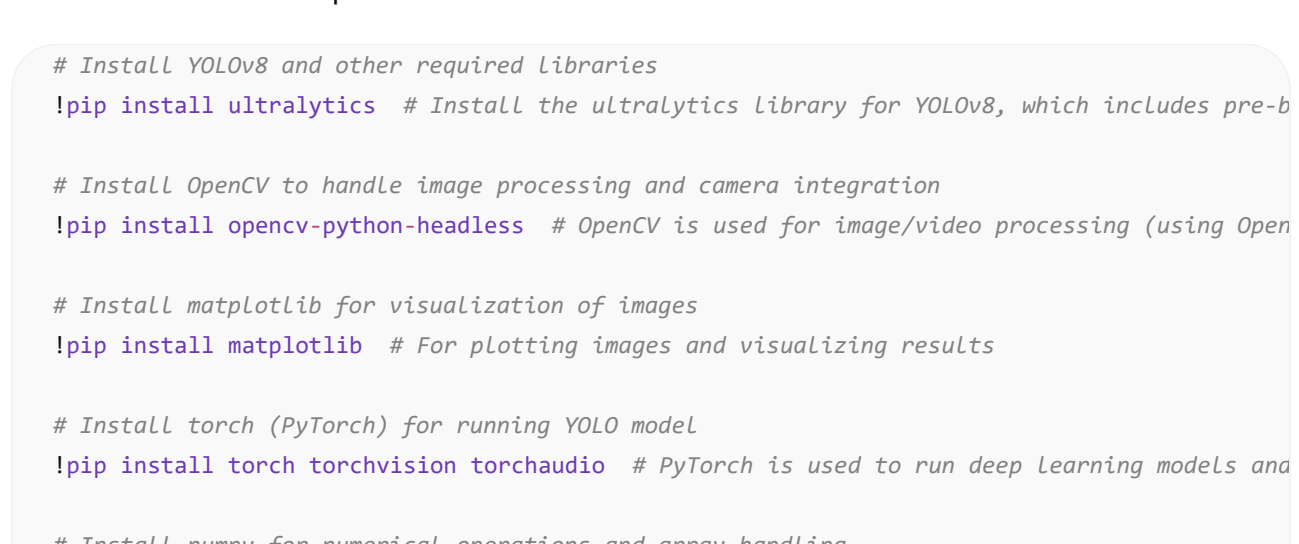
Your first task is:

Collect or prepare a labelled shuttlecock detection dataset.

Minimum dataset target:



That is your starting point.



Yes, **you can run the project on your laptop with its camera**, and your code can **detect the shuttlecock** using the camera module, along with the simulated environment.

Here's how:

1. Camera Module Integration:

Your laptop's camera will act as the real-world sensor for shuttlecock detection. You'll use libraries like **OpenCV** to capture video frames from the camera.

2. Object Detection Model:

You will train the **YOLOv8 model** (or another object detection model) on your shuttlecock dataset. Once trained, you can use this model to predict the shuttlecock's location in real-time from the camera feed.

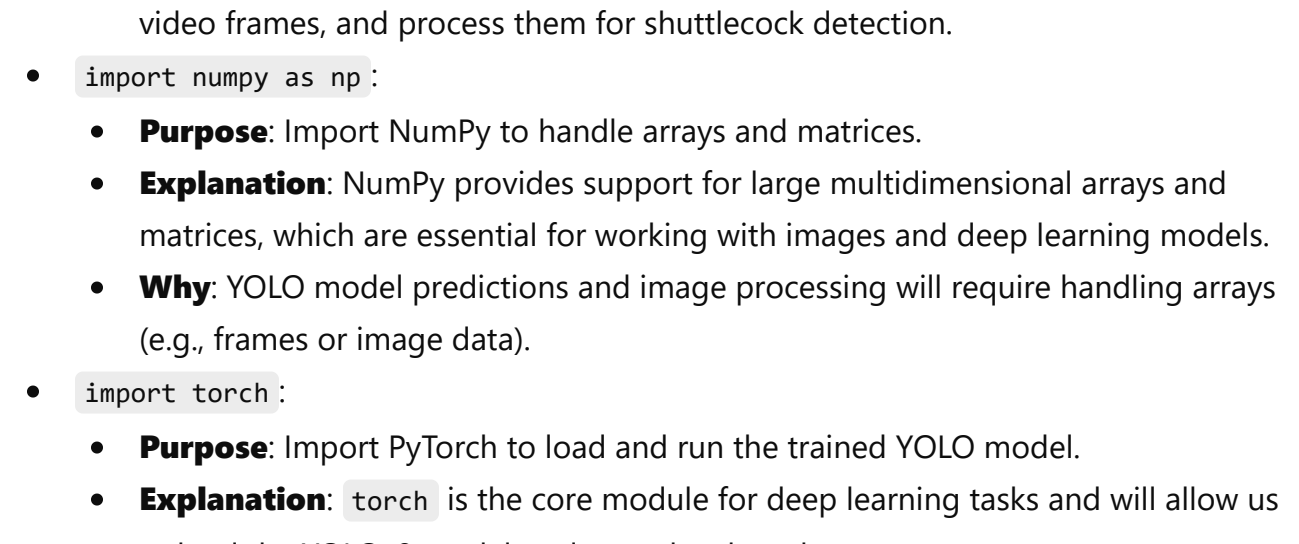
3. Simulation:

At the same time, for the **simulation** part, you can integrate a virtual environment (e.g., using **Gazebo** or **Unity**) to test how the model would behave in a controlled setup. The **simulated** environment will mimic real-world conditions for shuttlecock detection.

Workflow:

1. **Train the YOLO model** on a shuttlecock detection dataset (supervised learning).
2. **Use OpenCV** to capture video from your laptop's camera.
3. **Feed camera frames to the trained YOLO model** for shuttlecock detection in real-time.
4. **Run simulations** to test how the robot would pick up the shuttlecock, but the detection can happen using your camera.

So yes, the model can detect the shuttlecock using your laptop's camera in real-time, while also simulating the robot's movement.



Yes, you can create **perfect simulations** in **Python**, and both **Colab** and **VS Code** are viable options, depending on your needs.

1. Simulations in Python:

- **Simulation Environment:** You can use simulation environments like **PyBullet**, **Gazebo**, or **Unity** (with Python bindings) for building and running simulations in Python.

- **PyBullet** is lightweight and easy to use for robotics and physics simulations.
- **Gazebo** provides a more sophisticated physics simulation, but you'll need to set it up alongside Python.
- **Unity** with **ML-Agents** can be used for high-quality 3D simulations with Python.

These frameworks allow you to:

- Simulate the robot moving in a 3D environment.
- Detect objects (like shuttlecocks) using the camera and sensors.
- Interact with the virtual world (move the robot, pick up shuttlecocks, etc.).

2. Colab vs. VS Code:

Colab:

- **Pros:**
 - Ideal for **training ML models** because it provides free access to GPUs.
 - Easy to set up and get started with machine learning.
 - Useful for **data preprocessing** and experimentation in Python, especially when you need computational power.

- **Cons:**
 - **Limited simulation capabilities:** While you can run some simple simulations on Colab, it's not ideal for real-time robotics simulations (like with Gazebo or Unity).
 - Cannot easily connect to your **laptop's camera** for live feed processing unless you upload your images manually.

VS Code:

- **Pros:**
 - Full control over your development environment.
 - You can run **real-time simulations** using frameworks like PyBullet, Gazebo, or Unity.
 - Easily integrate your **laptop's camera** to capture live data from the camera and use it for object detection.
 - Supports **debugging**, **version control**, and all your project files.

- **Cons:**
 - **No GPU support** for training large ML models unless you have a local GPU or use cloud services like **Azure**, **AWS**, or **Google Cloud**.

My Recommendation:

- **Start with Colab to train your ML model** for shuttlecock detection.
 - **Train your YOLO or other object detection models** here since it's optimized for ML tasks, and you can use free GPU/TPU.

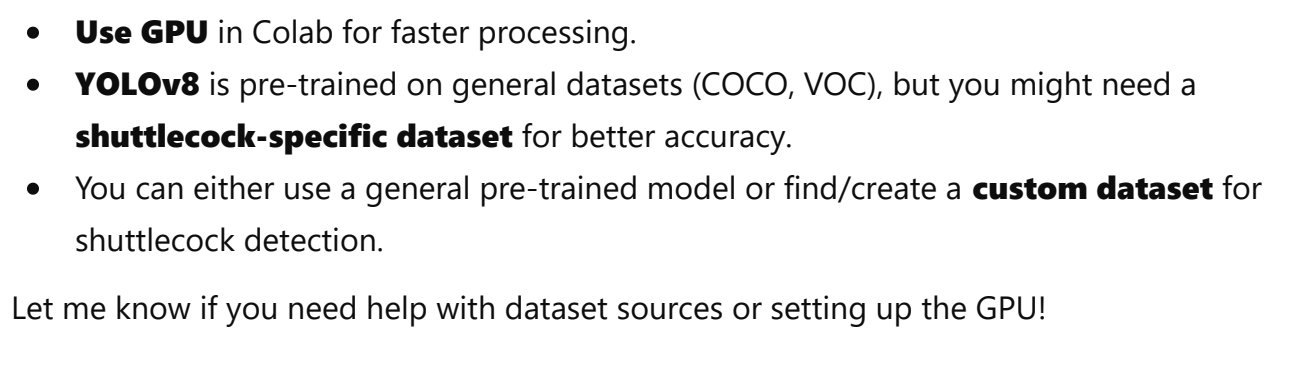
- **Use VS Code for Simulations:**
 - Once you have a trained model, you can move to **VS Code** to run the robot **simulation with real-time camera feed** and work on integrating **PyBullet/Gazebo/Unity**.
 - VS Code will allow you to integrate both **simulation environments** and **real-time camera feed** from your laptop.

Final Step-by-Step Process:

1. **In Colab:**
 - Train your **YOLO model** (or any object detection model) on the shuttlecock dataset.
 - Test the model and export the trained model weights.

2. **In VS Code:**
 - Set up a **simulation environment** using **PyBullet**, **Gazebo**, or **Unity**.
 - Capture real-time video using your laptop's camera via **OpenCV**.
 - Use the trained model to detect shuttlecocks from the camera feed in the simulation.
 - Use the model to control the robot in the simulation (e.g., move towards shuttlecock, pick it up).

In short, **Colab for training** and **VS Code for simulations and real-time control** is the best workflow for your project.



Alright, let's begin with **setting up the environment on Google Colab** for your project, including detailed comments. We will install the necessary libraries, set up the environment, and load your dataset.

Step 1: Setup Google Colab Environment

1. Install Necessary Libraries

We'll need several libraries to handle object detection and image processing. Here's how to set them up:

Detailed Comments:

Code Explanation:

- **!pip install ultralytics:**
 - **Purpose:** Install **YOLOv8** (from the ultralytics package) directly into Colab.
 - **Explanation:** The **ultralytics** library provides an easy-to-use interface for training and using YOLOv8 models (popular for object detection).
 - **Why:** We need this for shuttlecock detection using YOLOv8, which will process video frames from the camera to detect shuttlecocks.
- **!pip install opencv-python-headless:**
 - **Purpose:** Install OpenCV without GUI support (for working in Colab).
 - **Explanation:** OpenCV is used for image and video processing. In Colab, we do not need GUI functionalities, so we use the headless version (**opencv-python-headless**).
 - **Why:** We will use OpenCV to process the real-time camera feed from your laptop (or simulation) to detect shuttlecocks in each frame.
- **!pip install matplotlib:**
 - **Purpose:** Install matplotlib for plotting images in notebooks.
 - **Explanation:** Matplotlib is a plotting library that helps visualize images, graphs, and data.
 - **Why:** We need to visualize the detected shuttlecocks during object detection to confirm our model's performance.
- **!pip install torch torchvision torchaudio:**
 - **Purpose:** Install PyTorch and related packages for deep learning.
 - **Explanation:** PyTorch is a popular framework for building and running deep learning models. **torchvision** provides tools for handling image data, and **torchaudio** is for audio data (not used here, but included by default).
 - **Why:** YOLOv8 is built on PyTorch, and we need this to run the trained model for shuttlecock detection.
- **!pip install numpy:**
 - **Purpose:** Install NumPy for numerical operations.
 - **Explanation:** NumPy is used for handling arrays and matrices, which are crucial for image processing and model computations.
 - **Why:** The YOLO model requires array manipulations to process images and generate predictions.

Step 2: Import Libraries in Code

After installing the necessary libraries, let's import them into the code to be used later.

Detailed Comments:

Code Explanation:

- **import cv2:**
 - **Purpose:** Import the OpenCV library for image and video processing.
 - **Explanation:** **cv2** is the OpenCV module used for capturing video streams, processing images, detecting edges, etc.
 - **Why:** We will use OpenCV to interact with the camera (or simulation), capture video frames, and process them for shuttlecock detection.
- **import numpy as np:**
 - **Purpose:** Import NumPy to handle arrays and matrices.
 - **Explanation:** NumPy provides support for large multidimensional arrays and matrices, which are essential for working with images and deep learning models.
 - **Why:** YOLO model predictions and image processing will require handling arrays (e.g., frames or image data).
- **import torch:**
 - **Purpose:** Import PyTorch to load and run the trained YOLO model.
 - **Explanation:** **torch** is the core module for deep learning tasks and will allow us to load the YOLOv8 model to detect shuttlecocks.
 - **Why:** YOLOv8 is built on PyTorch, and we'll use this library to run inference on images or camera frames.
- **import matplotlib.pyplot as plt:**
 - **Purpose:** Import matplotlib to visualize the detection results.
 - **Explanation:** **plt** is used to plot graphs, images, or any other data. We'll use it to visualize the camera feed with detected shuttlecocks.
 - **Why:** It's important to display the images with predictions to ensure that the model detects shuttlecocks correctly.
- **from ultralytics import YOLO:**
 - **Purpose:** Import YOLO from the **ultralytics** library.
 - **Explanation:** This imports the YOLOv8 model and utilities that come pre-trained for object detection tasks.
 - **Why:** We'll use this to load a pre-trained YOLOv8 model for detecting shuttlecocks in camera frames.

Step 3: Load YOLOv8 Model

Now, let's load the YOLOv8 model. This is an essential step before performing object detection.

Detailed Comments:

Code Explanation:

- **model = YOLO('yolov8n.pt'):**
 - **Purpose:** Load a pre-trained YOLOv8 model.
 - **Explanation:** **YOLO('yolov8n.pt')** loads a pre-trained model (YOLOv8 nano in this case, which is lightweight and faster). You can change this to other YOLO variants based on your accuracy and performance requirements.
 - **Why:** We need the pre-trained model to detect shuttlecocks in images. This model has been trained on a large dataset and can generalize well to new images, including those of shuttlecocks.

Step 4: Set Up Camera Feed (Using Colab's Webcam)

Now, let's capture the camera feed from your laptop. Since you are using **Colab**, you can use a simulated webcam or upload your dataset for testing. However, for real-time webcam testing, I would recommend running this step in **VS Code** (locally) since Colab has limitations for webcam access.

Detailed Comments:

Code Explanation:

- **cap = cv2.VideoCapture(0):**
 - **Purpose:** Initialize the webcam feed.
 - **Explanation:** **cv2.VideoCapture(0)** is used to access the webcam. The parameter **0** refers to the default camera on your laptop. If you have multiple cameras, you can change this index to **1**, **2**, etc., depending on the camera you want to access.
 - **Why:** This line starts the webcam feed so that the robot can capture real-time video frames, which will be fed into the YOLO model for shuttlecock detection.

Next step:

After this, we can proceed to processing the frames from the camera and detecting shuttlecocks. Would you like to continue with this?

GPU Setup on Colab:

Yes, we **can use GPU** in Colab for faster model inference and processing.

To enable GPU in Colab:

1. Go to **Runtime > Change runtime type**.
2. Select **GPU** under the **Hardware accelerator** option.
3. This will enable GPU support for training and inference.

Dataset:

- **YOLOv8 Model:** The **YOLOv8 model** we're using comes **pre-trained** on a large dataset (like COCO or VOC), which allows it to detect general objects such as people, cars, etc. However, **shuttlecocks** may not be in the pre-trained dataset.

Option 1: Use Pre-trained YOLOv8 Model

- If the pre-trained YOLOv8 model is sufficient (it may work decently for shuttlecock detection if trained on similar objects), you can start with that.

Option 2: Get a Shuttlecock Dataset:

- **Search for a shuttlecock dataset** on platforms like **Kaggle**, **Roboflow**, or other dataset repositories. Look for an object detection dataset with shuttlecock labels.
 - You can also create your own dataset by taking images and labeling them using tools like **Labelling** or **Roboflow**.

Summary:

- **Use GPU** in Colab for faster processing.
- **YOLOv8** is pre-trained on general datasets (COCO, VOC), but you might need a **shuttlecock-specific dataset** for better accuracy.
- You can either use a general pre-trained model or find/create a **custom dataset** for shuttlecock detection.

Let me know if you need help with dataset sources or setting up the GPU!